



Sunbeam Institute of Information Technology

Pune and Karad

Module – Real Time Operating System

Trainer - Devendra Dhande

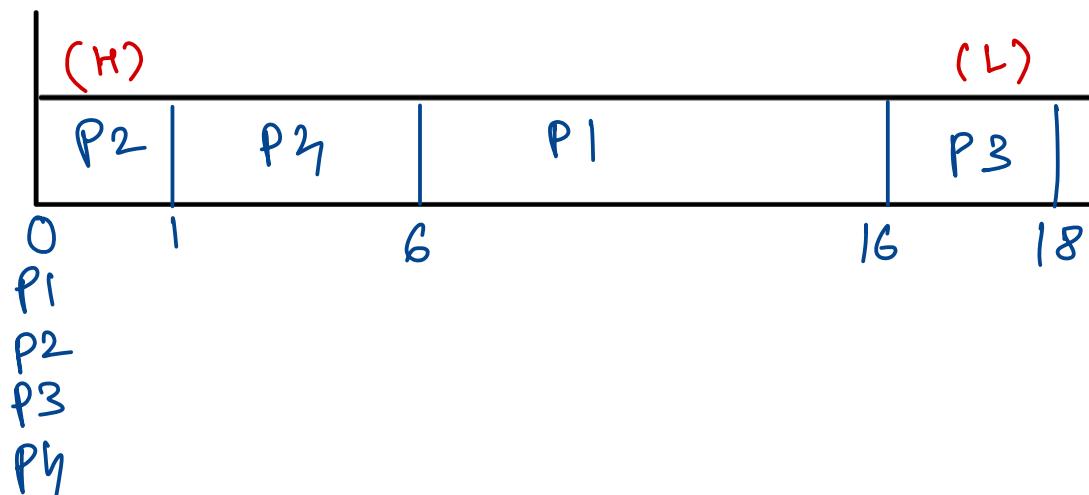
Email – devendra.dhande@sunbeaminfo.com

Priority

(Non-preemptive)

Process	Arrival	CPU Burst	Priority
P1	0	10	3
P2	0	1	1 (H)
P3	0	2	4 (L)
P4	0	5	2

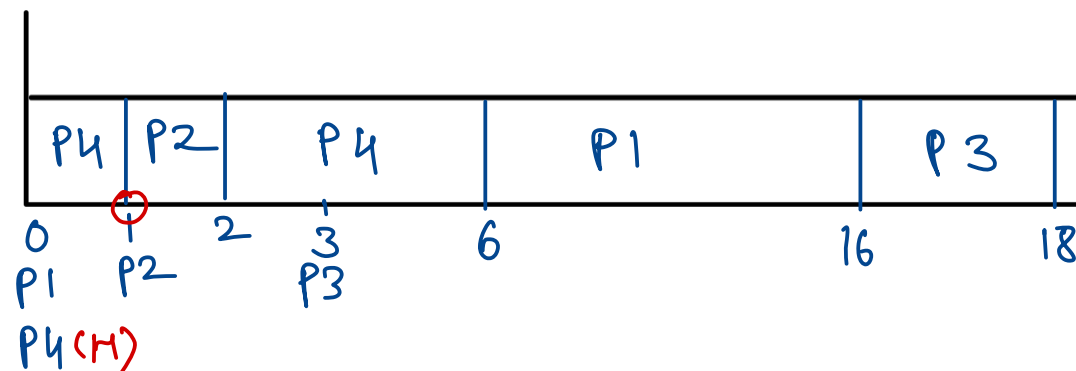
WT	RT	TAT
6	6	16
0	0	1
16	16	18
1	1	6

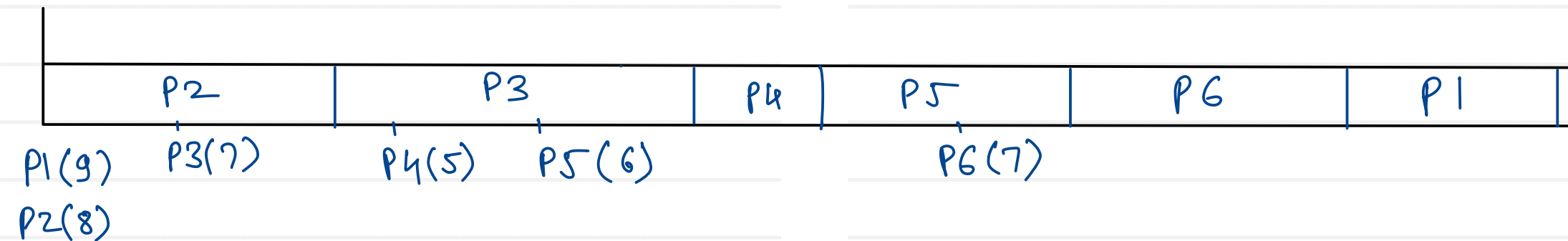


(preemptive)

Process	Arrival	CPU Burst	Priority
P1	0	10	3
P2	1	1	1
P3	3	2	4
P4	0	5	2

WT	RT	TAT
6	6	16
0	0	1
13	13	15
1	0	6





Starvation :

- Due to low priority, process do not get enough CPU time to execute
- need to wait for longer duration in ready queue to get CPU access.

Aging :

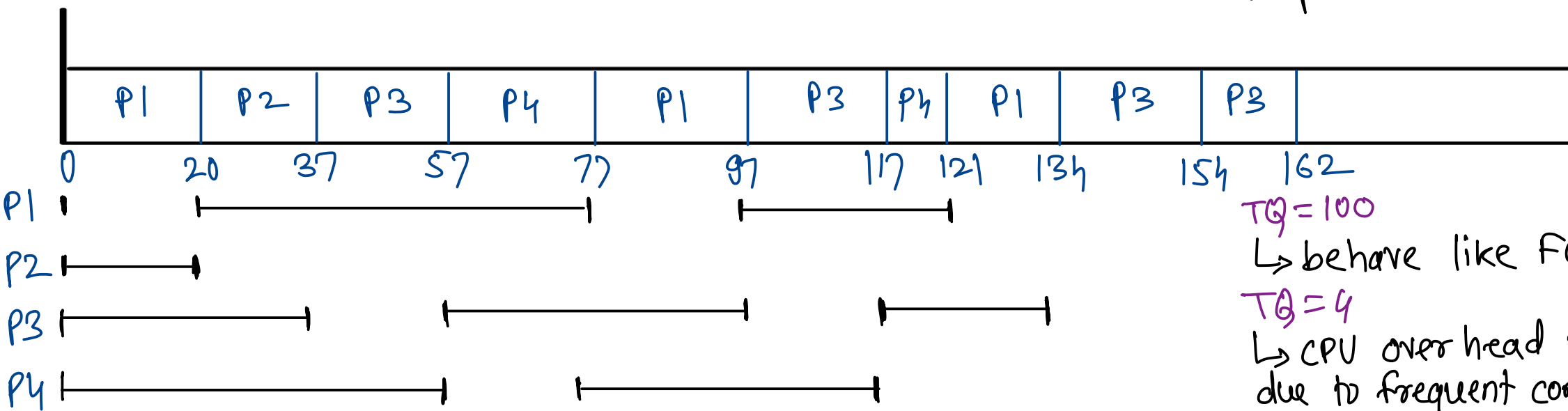
- increase the priority of process (who is not getting chance to execute) gradually.

RR (Round Robin) (preemptive)

Process	CPU Burst
P1	53
P2	17
P3	68
P4	24

WT RT
 33, 13, 0 0 + 57 + 24 0
 0 20 20
 48, 28, 8, 0 37 + 40 + 17 37
 4, 0 57 + 10 57

- CPU time is divided into multiple time slices, is called time quantum.
- for every time quantum next process is scheduled.
- Time quantum = 20
- first waiting time is always response time.



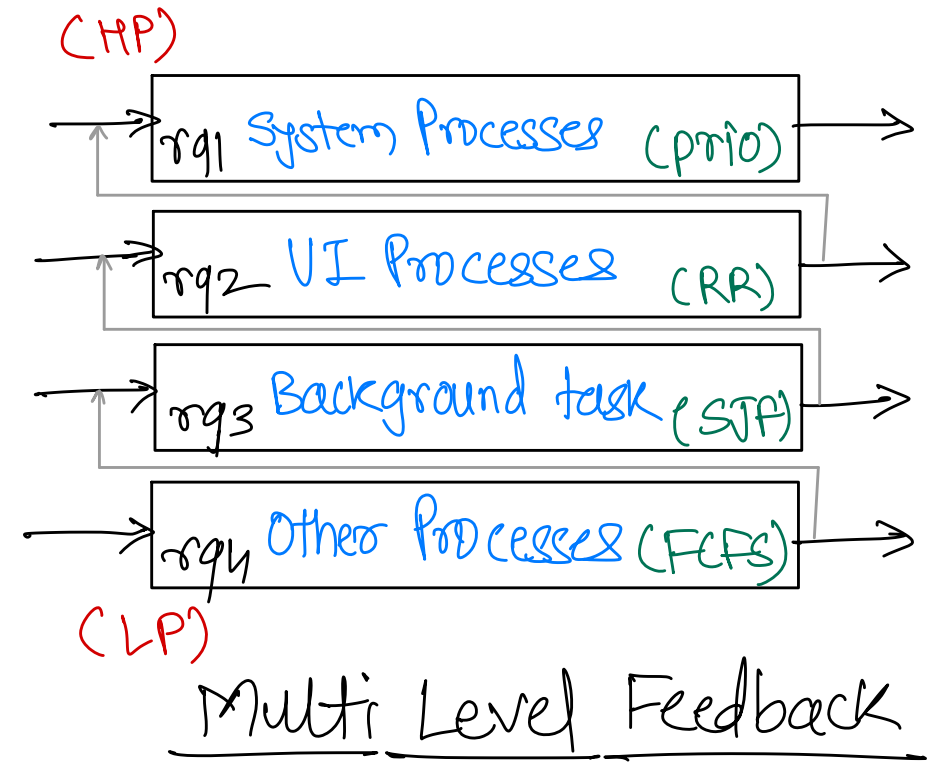
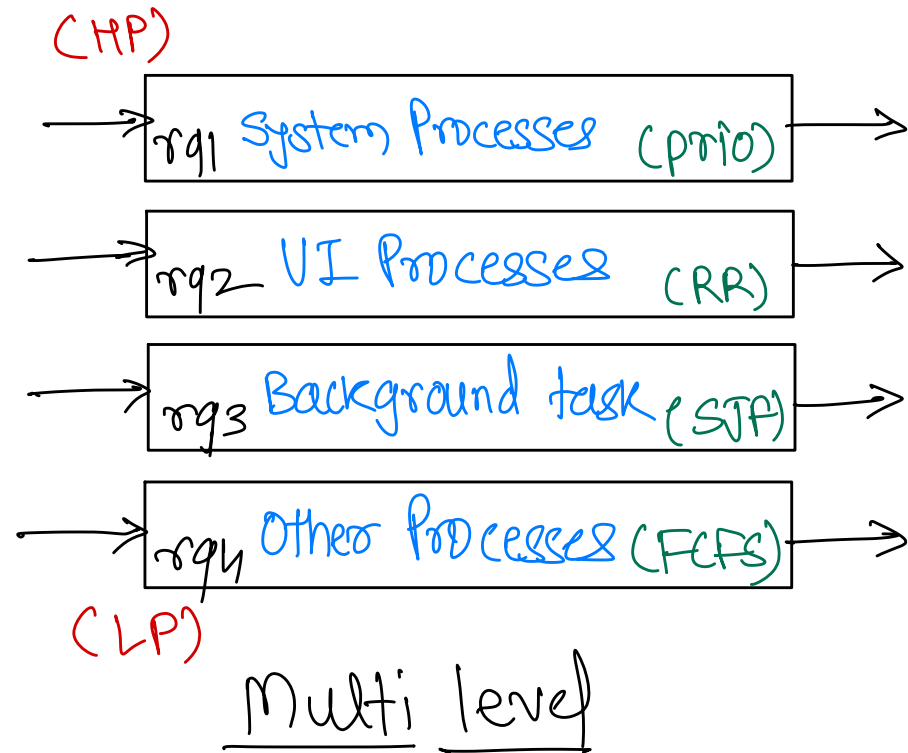
TQ = 100

↳ behave like FCFS

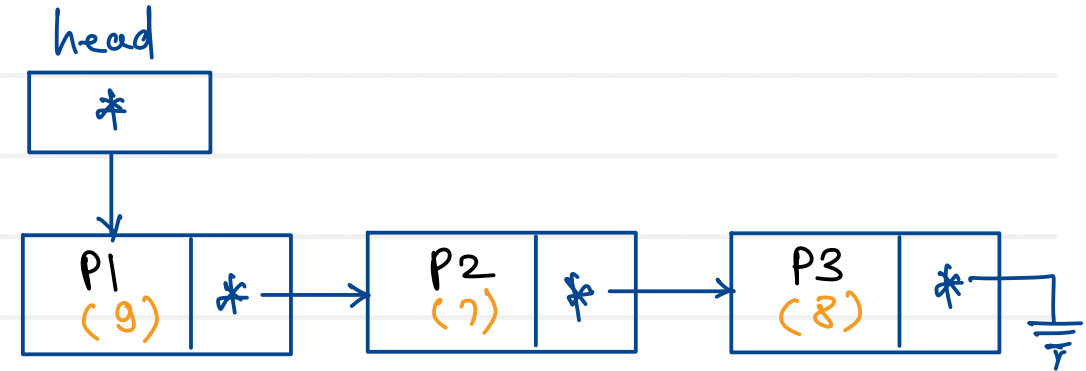
TQ = 4

↳ CPU over head will increase due to frequent context switching

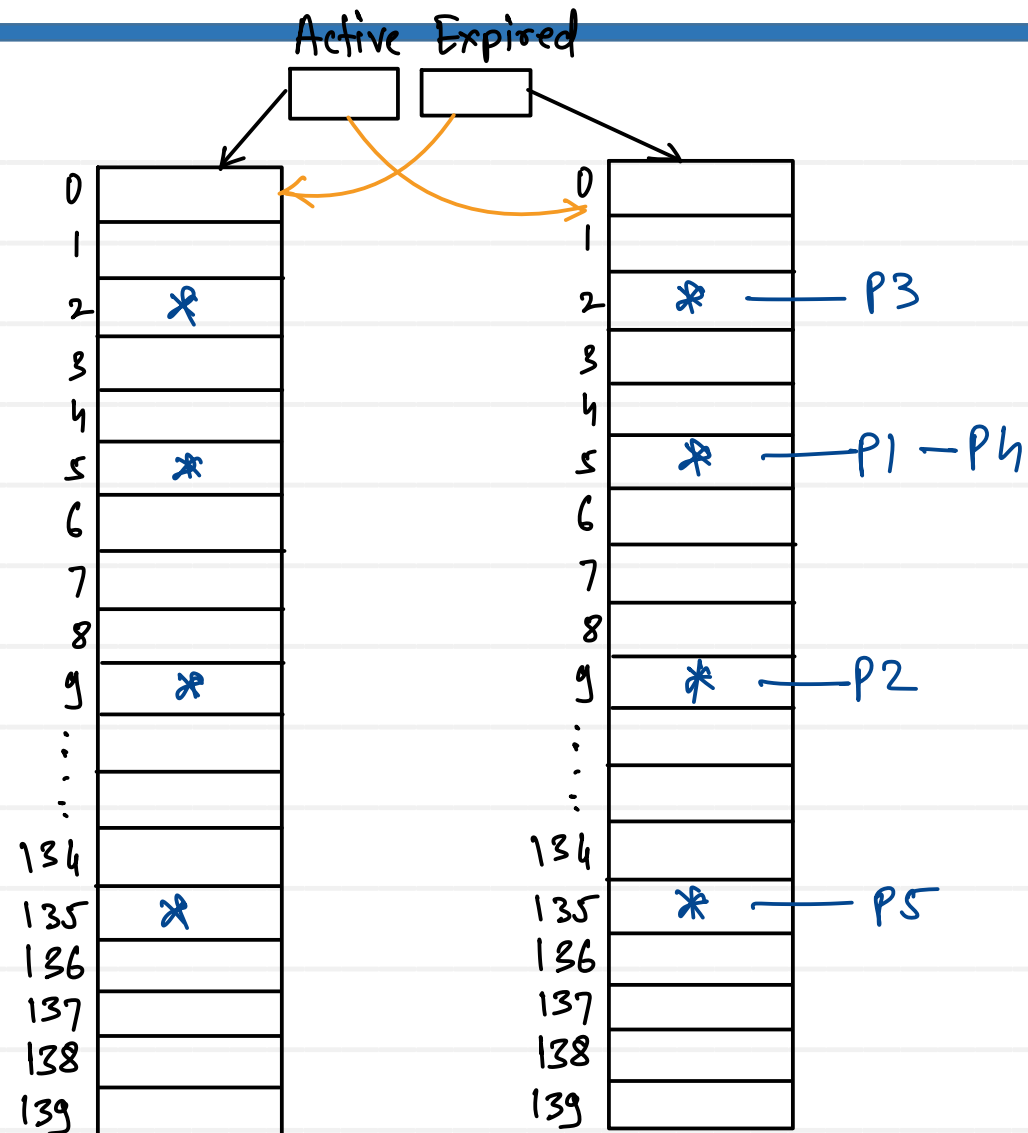
Multi Level Ready Queue



- All ready processes are kept into single linked list, which is called as "run queue"
- To choose high priority which will execute next on CPU, need to traverse whole linked list.
- Time complexity for this operation is $O(n)$ that's why this scheduler is also called as $O(n)$ scheduler



- Two run queues are created to keep processes.
 1. Active run queue
 2. Expired run queue
- size of both queues is equal to max priority level.
- Array of fixed size is used to create both queues & array indices indicates priority levels.
- processes are added into corresponding queue of active array. & whenever their time slice will finish, move into expired array.
- when active array will become empty, expired array will become active & active array will become expired.

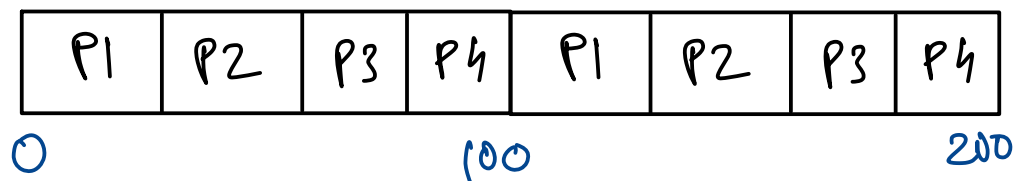
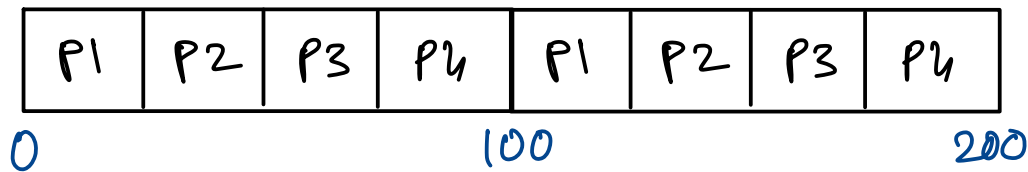


- CPU time is divided into time slices (epoch) (targetted latency)
- some share of each epoch is given to the processes which are in ready queue.
- share is given to the process on the basis of their priority ← high – more share
low – less share
- priority of every process is decided by its nice value
- nice values range ---> -20 to +19 (40 values)
 - * -20 - highest priority
 - * +19 - lowest priority

Process	Nice Value
P1	10
P2	10
P3	10
P4	10

Epoch - 100

Process	Nice Value
P1	5
P2	5
P3	10
P4	10



Process	Nice value
P1	0
P2	0
P3	0
P4	0



Targetted Latency (TL) = 100 ms

$$TS_{ni} = \frac{W_{ni}}{\sum W_x} \times TL$$

$$n_i = 0, \quad W_0 = 1024$$

$$TS_0 = \frac{W_0}{W_0 + W_0 + W_0 + W_0} \times 100$$

$$= \frac{1024}{1024 + 1024 + 1024 + 1024} \times 100$$

$$= 25 \text{ ms}$$

CFS - Time share calculation

Targetted Latency (TL) = 100 ms

$$TS_{ni} = \frac{W_{ni}}{\sum W_x} \times TL$$

Process	Nice value
P1	0
P2	5
P3	5
P4	5

$$n_i = 0, W_0 = 1024$$

$$n_i = 5, W_5 = 335$$

$$TS_0 = \frac{W_0}{W_0 + W_5 + W_5 + W_5} \times 100$$

$$TS_5 = \frac{W_5}{W_0 + W_5 + W_5 + W_5} \times 100$$

$$= \frac{1024}{1024 + 335 + 335 + 335} \times 100$$

$$= \frac{335}{1024 + 335 + 335 + 335} \times 100$$

$$= 50.46 \text{ ms}$$

$$= 16.51 \text{ ms}$$

$$P1 = 50.46$$

$$P2, P3, P4 = 16.51$$



0

100

Targetted Latency (TL) = 100 ms

$$TS_{ni} = \frac{W_{ni}}{\sum W_x} \times TL$$

Process	Nice value
P1	0
P2	0
P3	5
P4	5
P5	5

$$n_i = 0, W_0 = 1024$$

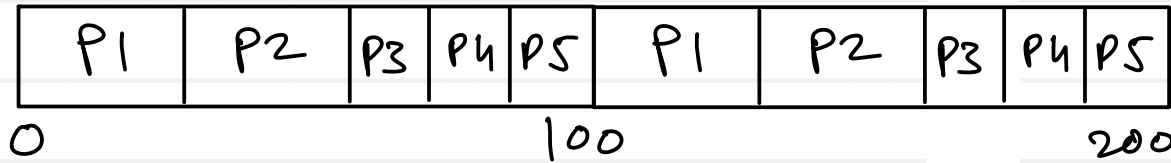
$$n_i = 5, W_5 = 335$$

$$\begin{aligned}
 TS_0 &= \frac{W_0}{W_0 + W_0 + W_5 + W_5 + W_5} \times 100 \\
 &= \frac{1024}{1024 + 1024 + 335 + 335 + 335} \times 100 \\
 &= 33.54 \text{ ms}
 \end{aligned}$$

$$\begin{aligned}
 TS_5 &= \frac{W_5}{W_0 + W_0 + W_5 + W_5 + W_5} \times 100 \\
 &= \frac{335}{1024 + 1024 + 335 + 335 + 335} \times 100 \\
 &= 10.97 \text{ ms}
 \end{aligned}$$

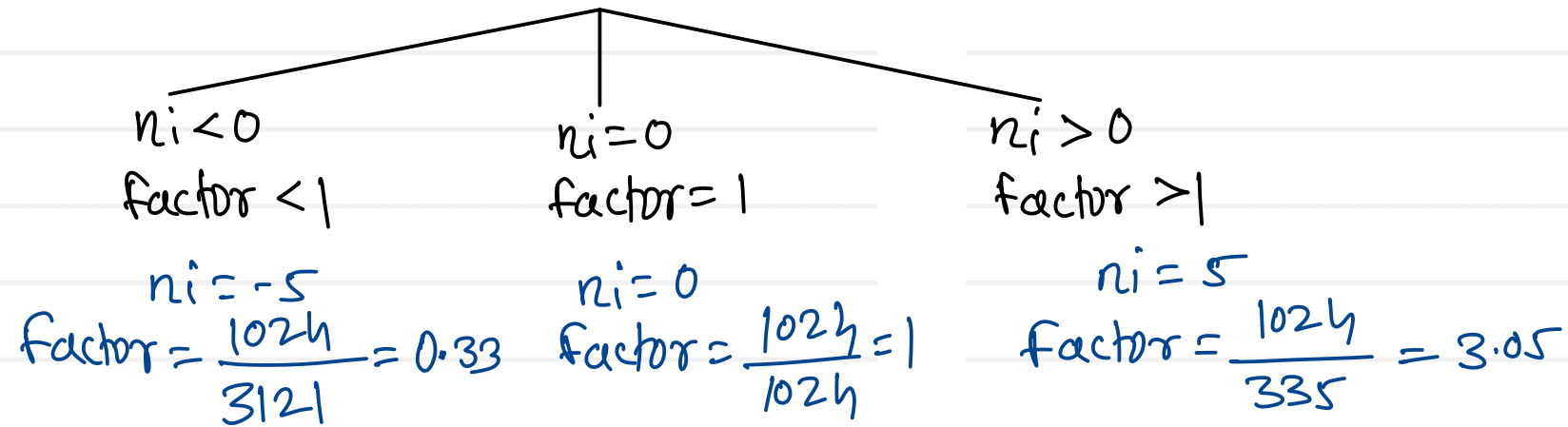
$$P1, P2 = 33.54$$

$$P3, P4, P5 = 10.97$$



$$V_{\text{runtime}} = \text{factor} * A_{\text{runtime}}$$

Where $\text{factor} = \frac{W_0}{W_i}$, $A_{\text{runtime}} = \text{CPU burst/time}$



$$A_{\text{runtime}} = 20 \text{ ms}$$

$$V_{\text{runtime}} = 0.33 \times 20 = 6.6$$

$$V_{\text{runtime}} = 1 \times 20 = 20$$

$$V_{\text{runtime}} = 3.05 \times 20 = 60.15$$

1) Real-time policy :

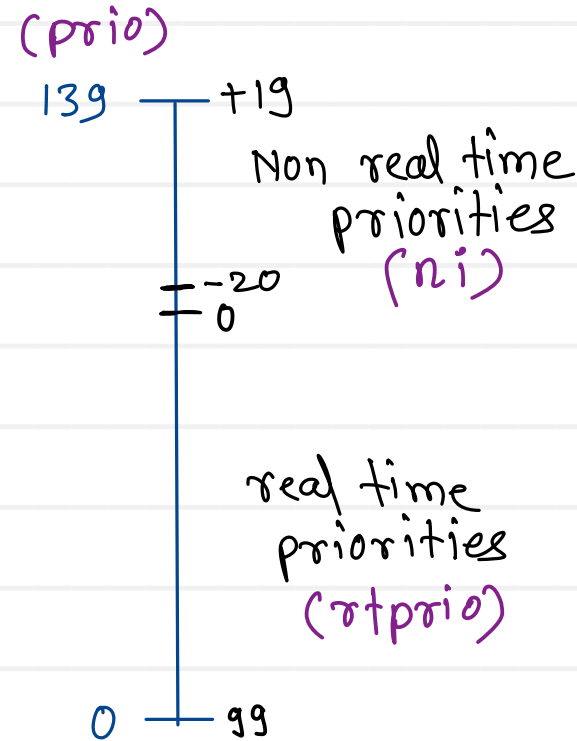
- high priority process will not wait for low priority process. (L) (H)
- real-time priorities - 0 to 99

1. SCHED_FIFO (FCFS)
2. SCHED_RR (RR)

2) Non real-time policy :

- high priority process will wait for low priority process (H) (L)
- non real-time priorities - -20 to +19

3. SCHED_OTHER (TS)
4. SCHED_BATCH (TS)
5. SCHED_IDLE (TS)



- processes of real time classes has higher priority than processes of non real time classes.

- all non real time priorities are arranged after real time priorities

- non real time priorities are mapped from 100 to 139



Thank you!!!

Devendra Dhande

devendra.dhande@sunbeaminfo.com